

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI THỰC HÀNH
BÀI 3: BỘ CÂN BẰNG ÂM THANH (AUDIO EQUALIZED)

GVHD: Nguyễn Tuấn Hùng

STT	Họ và tên	MSSV
1	Lâm Trọng Nhân	2312423
2	Nguyễn Gia Minh	2312084
3	Đỗ Hữu Khánh Duy	2310455

PHÂN CÔNG NHIỆM VỤ

Họ và tên	MSSV	Nhiệm vụ	Hoàn thành	Ký tên
Lâm Trọng Nhân	2312423	Bộ lọc, thu, phát tín hiệu âm thanh	100%	
Nguyễn Gia Minh	2312084	Giao thức Uart, giao diện người dùng	100%	
Đỗ Hữu Khánh Duy	2310455	Xử lý, đồng bộ xung nhịp clock	100%	

Mục lục

Chương I. THIẾT KẾ HỆ THỐNG EQUALIZER 10 BĂNG TẦN TRÊN FPGA	4
1.1 Giới thiệu tổng quan về hệ thống <i>Graphic Equalizer</i>	4
1.2 Thiết kế bộ số bộ lọc FIR cho 10 dải tần số bằng <i>MATLAB</i>	5
1.3 Cài đặt lõi xử lý <i>eq10_core</i> : Bộ lọc FIR và Bộ nội suy hệ số khuếch đại	7
Chương II. GIAO TIẾP MÁY TÍNH VÀ XỬ LÝ ÂM THANH	9
2.1 Giới thiệu chuẩn giao tiếp <i>UART</i> và module truyền/nhận (<i>uart_rx, uart_tx</i>)	9
2.2 Bộ chia tần số	10
2.3 Cập nhật thông số <i>EQ</i> , <i>Volume</i> và <i>Gain</i> theo thời gian thực (bằng <i>I2C</i>)	11
Chương III. KIẾN TRÚC TỔNG THỂ VÀ TRIỂN KHAI TRÊN KIT DE2	15
3.1 Sơ đồ kết nối tổng quát toàn hệ thống (<i>Top-level lab3_eq10_uart</i>)	15
3.2 Sơ đồ khối của các module thành phần	17
3.2.1 Khối nhận và giải mã lệnh <i>Uart</i> (<i>FSM_Decoder</i>)	17
3.2.2 Khối xử lý dải âm (<i>eq10_core</i>)	18
3.2.3 Khối giao tiếp chip âm thanh (<i>wm8731_out_runtime_vol_gain</i>)	19
3.3 Cấu hình tài nguyên và gán chân tín hiệu trên kit <i>Altera DE2</i>	19
Chương IV. KẾT QUẢ THỰC NGHIỆM VÀ NHẬN XÉT	21
4.1 Kết quả biên dịch và phân tích tài nguyên phân cứng	21
4.4 Nhận xét hạn chế và hướng phát triển hệ thống	22

Chương I. THIẾT KẾ HỆ THỐNG EQUALIZER 10 BĂNG TẦN TRÊN FPGA

1.1 Giới thiệu tổng quan về hệ thống Graphic Equalizer

Hệ thống cân bằng âm thanh (Graphic Equalizer) là thiết bị hoặc là 1 phần mềm với công dụng điều chỉnh biên độ âm thanh bằng các hiệu chỉnh ứng với một giá trị tần hoặc băng tần nhất định, trực quan hóa hình dạng tần số.

Trong bài Lab này nhóm chúng em tìm hiểu về một hệ thống chia dải tần số âm thanh trong khoảng 20-20 (kHz), đây là khoảng tần số con người nghe được, tiến hành thực hiện mô phỏng lại bộ cân bằng âm thanh này với mục đích điều chỉnh độc lập biên độ của từng dải tần một cách chủ động, và thay đổi ngõ ra âm thanh theo thời gian thực. Điều này được thực hiện bằng FPGA trong dải âm thanh tai người có thể nghe thấy để xác thực tính đúng của đề tài.

```
wire signed [DATA_W-1:0] y0, y1, y2, y3, y4, y5, y6, y7, y8, y9;
wire v0, v1, v2, v3, v4, v5, v6, v7, v8, v9;
fir_filter_stream    #(.DATA_W(DATA_W),    .COEF_W(COEF_W),    .NTAPS(NTAPS),
.COEFF_FILE(COEFF0_FILE)) u_fir0 (
    .clk(clk),    .rst_n(rst_n),    .sample_en(sample_en),    .dataIn(sample_in),    .dataOut(y0),
.dataOut_valid(v0));
...
fir_filter_stream    #(.DATA_W(DATA_W),    .COEF_W(COEF_W),    .NTAPS(NTAPS),
.COEFF_FILE(COEFF9_FILE)) u_fir9 (
    .clk(clk),    .rst_n(rst_n),    .sample_en(sample_en),    .dataIn(sample_in),    .dataOut(y9),
.dataOut_valid(v9));
```

Ở đây ta sử dụng hàm `fir_filter_stream #(...)` lại ở Lab 2 nhưng ta dùng ký tự `#` để ghi đề thông số ở file cũ để tiến thành nạp thông số mới để cho ra tần số mong muốn với ngõ ra ở wire `y[0:9]`

1.2 Thiết kế bộ số bộ lọc FIR cho 10 dải tần số bằng MATLAB

Nhóm chúng em sử dụng bộ lọc FIR pha tuyến tính (đối xứng qua hệ số giữa) để đảm bảo rằng không bị méo pha âm thanh. Bởi vì tất cả âm thanh đều bị trễ một khoảng thời gian bằng 1 nửa chu kỳ bộ lọc (tương đối nhỏ) nên tai người không thể nghe thấy được.

Bộ lọc FIR pha tuyến tính cần được biên dịch trên phần mềm MATLAB, vì đây là chương trình đạt một số tiêu chuẩn hơn về tính toán hệ số, phù hợp để tìm hệ số trong bài Lab này, với chương trình tìm hệ số tương tự như trên bài Lab2 nhưng thay vào đó ở lần này ta sẽ phải tìm hệ số cho cả 10 dải tần số với 127 hệ số (taps). Đây là điểm số lượng hợp lý cân bằng giữa đánh đổi độ trễ và độ chính xác của ngõ ra. Vừa cân bằng tài nguyên và đảm bảo ngõ ra với độ dốc cắt tần số đủ chính xác.

Trong MATLAB tương tự như LAB 2 ta có những thông số:

Tham số	Giá trị	Ý nghĩa
Fs	48000 Hz	Tần số lấy mẫu
NTAPS	127	Số lượng hệ số.
COEF_W	18 bit	Định dạng hệ số trong tính toán (Q1.17).

Với 10 ngõ ra là 10 bộ hệ số được lưu với tên định dạng: fir-XXkhz.mem. Trong MATLAB có:

```
for k = 1:numel(bands)
    b = bands{k};
    switch lower(b.type)
        case ' 'nếu thông số tùy chỉnh thuộc dải tần số nào thì nó sẽ hiện ở đây ' '
            Wn = b.f2 / (Fs/2);
            if USE_KAISER
                win = kaiser(NTAPS, BETA)';
```

```

        h = fir1(NTAPS-1, Wn, "xx", win, 'scale');
    else
        h = fir1(NTAPS-1, Wn, "xx");
    end

```

Với “xx” là miền tần số có thông số mà chúng em tùy chỉnh ở trên thì sẽ chuẩn hóa tần số cắt theo chuẩn Nyquist ‘ $Wn = F_c / (F_s/2)$ ’ và tạo bộ lọc tương ứng với miền tần số đó, kết quả của hàm cửa sổ này là ‘h’ là 127 con số thực dạng thập phân. Vectơ ‘h’ này được lượng tử hóa về số nguyên 18bit – Fixed-point (nhân với 131072, sau đó làm tròn (round) bằng cách vớt bỏ đuôi thập phân)

```

% Scale sang signed fixed-point
scale = 2^(COEF_W-1) - 1;
hq = round(h * scale);
% Saturation tranh vượt giới hạn signed
minv = -2^(COEF_W-1);
maxv = 2^(COEF_W-1) - 1;
hq(hq < minv) = minv;
hq(hq > maxv) = maxv;

```

Khi đã có số INTEGER thì sẽ chuyển sang số HEX hệ cơ số 16 để đẩy hệ số đã lưu của riêng một tần số vào file .mem theo tên định dạng như trên.

Đó là thuật toán tìm hệ số của một tần số nhất định, cuối cùng chúng em thay vì làm lần lượt 10 lần để tìm được 10 file thì đã áp dụng thuật toán sau để MATLAB đẩy 10 file trong 1 lần chạy luôn:

```

bands = { struct('name','31hz', 'type','low', 'f1', 0, 'f2', 46.5)
    struct('name','62hz', 'type','band', 'f1', 46.5, 'f2', 88.0)
    ...
    struct('name','8khz', 'type','band', 'f1',5650.0, 'f2', 11300.0)
    struct('name','16khz', 'type','high', 'f1',11300.0,'f2', 0) };

```

Ở đây chúng em dùng hàm struct() và vòng lặp for để chạy 10 lần ra 10 file hệ số h luôn và mục đích của việc chia 10 dải tần số với tần số cắt trên của dải tần số bé sẽ là tần số cắt

dưới của dải tần số lớn kế tiếp của nó , điều này cho một bộ lọc 10 dải tần với không một khoảng thiếu tần số nào, khiến cho âm thanh không bị mất đi phần nào hết.

1.3 Cài đặt lõi xử lý eq10_core: Bộ lọc FIR và Bộ nội suy hệ số khuếch đại

Sau khi có được 10 dải tần số khác nhau từ quá trình lọc tuần tự 10 bộ lọc thì tiếp theo sẽ nhận giá trị từ máy tính để tiến hành làm việc với bộ nhân, để điều chỉnh biên độ trên một dải tần số xác định.

```
wire [15:0] g0, g1, g2, g3, g4, g5, g6, g7, g8, g9;
eq10_gain_lut u_g0(.db_in(db0), .gain_q2_14(g0));
...
eq10_gain_lut u_g9(.db_in(db9), .gain_q2_14(g9));
//
wire signed [39:0] p0 = $signed(y0) * $signed({1'b0, g0});
...
wire signed [39:0] p9 = $signed(y9) * $signed({1'b0, g9});
```

Module ‘eq10_gain_lut’ được cấu trúc bởi mục đích nhận mã là một vectơ 6 bit chứa giá trị [-12:+12], địa chỉ dải tần, và loại tín hiệu từ máy tính sau đó dùng bảng tra ‘Look up table’ để chuẩn hóa hệ số theo chuẩn Q2.14 (dấu chấm tĩnh), mục đích của bảng tra là để đỡ tốn tài nguyên của FPGA khi phải tính toán dấu chấm tĩnh, thay vào đó chúng em đã tiến hành chuyển đổi bên ngoài theo công thức $10^{\frac{-dB}{20}} \times 2^{14}$. Cùng với địa chỉ dải tần thì chúng em có được số theo chuẩn Q2.14 và nằm ở dải tần nào, vì chạy 1 lần 10 bộ luôn nên là không cần kiểm tra lại. Tiếp theo tiến hành nhân cùng lúc 10 giá trị vào ngõ ra 10 dải tần đã có ở trên:

```
wire signed [39:0] p0 = $signed(y0) * $signed({1'b0, g0});
...
wire signed [39:0] p9 = $signed(y9) * $signed({1'b0, g9});
// GAIN_SHIFT = 14 ; dịch phải lại để chuyển sang dạng số nguyên
wire signed [39:0] s0 = p0 >>> GAIN_SHIFT;
...
wire signed [39:0] s9 = p9 >>> GAIN_SHIFT;
```

‘\$signed’: nhân có dấu giá trị vừa quy đổi với bit 0 ở MSB để không bị chuyển sang số âm, rồi trước khi cộng tín hiệu lại 1 biến thì cần chuyển về lại dạng số đúng bằng cách dịch phải 14 lần ($\ll 14$). Khi cộng 10 tín hiệu 24 bit (sau khi nhân với giá trị điều chỉnh) thì dễ có hiện tượng ngõ ra vượt quá 24 bit (gây ra hiện tượng sai lệch tín hiệu) thì biến ‘sat24’ được lập ra để bảo vệ điều đó, tránh trường hợp đó xảy ra.

<pre>function [DATA_W-1:0] sat24; input signed [47:0] v; reg signed [47:0] vmax; reg signed [47:0] vmin; begin vmax = 48'sd8388607; vmin = -48'sd8388608;</pre>	<pre> if (v > vmax) sat24 = 24'sh7FFFFFFF; else if (v < vmin) sat24 = 24'sh8000000; else sat24 = v[23:0]; end endfunction</pre>
---	--

Việc ‘sat24’ chỉ lấy 24 bit LSB nhưng vẫn đảm bảo đầu ra âm thanh không bị thiếu, mất âm thanh, thì là do cấu trúc if_else đã lọc đi những trường hợp số quá bé hoặc quá lớn đấy đi rồi, nên là còn 24 bit cuối là chuẩn.

Chương II. GIAO TIẾP MÁY TÍNH VÀ XỬ LÝ ÂM THANH

2.1 Giới thiệu chuẩn giao tiếp UART và module truyền/nhận (uart_rx, uart_tx)

Để FPGA nhận được giá trị hiệu chỉnh trực tiếp từ máy tính thì chúng em sử dụng giao tiếp bằng giao thức Uart truyền, nhận nối tiếp các bit giữa FPGA và máy tính, trong bài lab này nhóm chúng em sử dụng kỹ thuật chống nhiễu khi nhận tín hiệu là đợi đến giữa chu kỳ (BAUD_DIV>>1) rồi mới đọc tín hiệu để tránh nhiễu xung.

Trong module nhận tín hiệu từ máy tính ‘uart_rx’ là một máy trạng thái với những trạng thái hoạt động như sau:

```
localparam IDLE = 2'd0,  
  
            START = 2'd1,  
  
            DATA = 2'd2,  
  
            STOP = 2'd3;
```

Trong trường hợp máy tính nhận được tín hiệu điều chỉnh biên độ với hệ số trên thanh trượt thì lập tức máy tính sẽ gửi đến theo dạng cấu trúc [‘loại’, ‘dải’, ‘giá trị’] với loại (volume, gain, band) ‘volume’ là điều chỉnh biên độ toàn dải tần, ‘gain’ là độ lợi ngõ vào âm thanh, ‘band’ là 10 dải tần mà đã chia từ trước. ‘dải’ nếu ta hiệu chỉnh giá trị nằm trong dải ‘band’ thì ở trong module “eq10_core” lập tức chuyển đến máy trạng thái và tiến hành nhân tuần tự 10 dải với những dữ liệu có trong 6 bit mà uart gửi đến. Giá trị mà uart gửi đến ở đây là giá trị số nguyên mà ta hiệu chỉnh có mức -12dB nên chúng em sử dụng thủ thuật thay đổi dải giá trị từ [-12:+12] => [0:24] để nhận sau đó chuyển về số có dấu(\$signed) khi trừ đi ‘6’d12’ để chuyển về giá trị cũ. Còn trong trường hợp ta chỉnh biên độ của 1 trong 2 thành phần ‘Volume’ hoặc ‘Gain’ thì ở module ‘eq10_core’ xem như không nhân gì cả mặc dù nó vẫn có nhân tuần tự nhưng vì ta không chỉnh hệ số nhân trên phần mềm C#. Thông số mà ta chỉnh thuộc 1 trong 2 thành phần kia sẽ chuyển đến module thực hiện công dụng chỉnh biên độ theo thời gian thực mà chúng em sẽ phân tích ở dưới.

Trong bài Lab này chúng em tìm hiểu thêm về bộ truyền lệnh với module ‘uart_tx’ giúp giao tiếp với máy tính 2 chiều, giao tiếp bằng chuỗi bit, có những trạng thái làm việc tương

tự như module ‘uart_rx’, mạch phát tín hiệu khi cờ báo ‘trigger’ được bật lên, khi ở trong quá trình gửi đi thì cờ báo bận ‘busy’ được kéo lên mức cao, sau đó dữ liệu đã được chuẩn bị sẵn ở trong dây ‘data_in’ được dẫn đến thanh ghi dịch, bước sang trạng thái gửi chuỗi bit với start_bit là bit 0, vì xung nhịp 50MHz của FPGA quá nhanh nên module chờ cho đến khi biến ‘baud_counter’ đến đến nửa chu kỳ rồi mới bắt đầu dịch 8 bit dữ liệu mang theo giá trị điều chỉnh biên độ ban này theo thứ tự LSB-MSB qua máy tính (tức là bit thứ [0] dịch trước), cuối cùng thì gửi bit stop [1] (cũng giữ mức 1 trong nửa chu kỳ để cho máy tính đọc được) và hạ cờ báo bận ‘busy’ xuống mức 0.

2.2 Bộ chia tần số

Bản thân FPGA cũng đã chạy với tốc độ xung nhịp là 50Mhz trong khi chip âm thanh trên kit DE2 Woflson wm8731 được lập trình trong module thu âm để hoạt động với xung nhịp khoảng 3.125MHz. Vì điều đó, nếu như ta không can thiệp gì thì sẽ xảy ra hiện tượng cờ báo đã sẵn sàng 1 mẫu âm thanh để được lấy đi từ wm8731 ‘left_valid’ sẽ báo lên trong 1 chu kỳ, ngay lúc đó FPGA chạy với xung 50MHz sẽ đọc được cái cờ báo này tầm 16 lần cái cờ báo này, và lấy được 16 cái mẫu âm thanh giống nhau, hậu quả là sẽ nghe được âm thanh lặp, giật ở ngõ ra. Để tránh tình trạng đó nhóm chúng em khắc phục bằng module ‘cdc_toggle_sync’ để cắt nhỏ cái cờ báo ‘left_valid’ đó xuống 1 xung chu kỳ nhiệm vụ ở 50MHz.

Nổi dây báo vào 1 biến register 3 bit _LSB để ổn định mức báo giữa 0-1 ở xung 50MHz (vẫn đang báo 16 lần), ở xung nhịp 50MHz dịch trái bit trong register và nạp ngõ ra ở nhịp sau vào LSB của thanh ghi dịch. tạo 1 dây ‘dst_toggle’ là ngõ ra của phép toán logic XOR của 2 bit MSB trong thanh ghi dịch tức là biến ‘dst_toggle’ chỉ lên mức 1 khi biến ngõ ra khác với biến ngõ vào của cờ báo ‘left_valid’, sau đó 16 chu kỳ xung thì ‘dst_toggle’ lại về mức logic [0], nên dây này đóng vai trò như là 1 cờ báo ở xung nhịp 50MHz.

<pre>reg src_toggle; always @(posedge src_clk or negedge src_rst_n) begin if (!src_rst_n) src_toggle <= 1'b0;</pre>	<pre>reg [2:0] sync_ff; always @(posedge dst_clk or negedge dst_rst_n) begin if (!dst_rst_n) sync_ff <= 3'b000; else sync_ff <= {sync_ff[1:0], src_toggle};</pre>
--	---

<pre> else if (src_pulse) src_toggle <= ~src_toggle; end </pre>	<pre> end </pre>
--	------------------

2.3 Cập nhật thông số EQ, Volume và Gain theo thời gian thực (bằng I2C)

Trong module này nhóm em sử dụng một số dây với chức năng như sau: biến dùng để lưu dữ liệu đã chuẩn bị đẩy vào chip âm thanh với mục đích cài đặt hệ thống (mux_input), dây báo ra lệnh cho giao thức I2C bắt đầu hoạt động (ignition), dây báo đẩy xong dữ liệu (finishi_flag) và (finish_d) là thanh register luôn trễ hơn biến báo xong của I2C. (hp_shadow) sẽ đóng vai trò lưu trữ dữ liệu mới nhận được từ bộ cài đặt thông số, I2C sẽ lấy dữ liệu mới ở đây với cờ báo là (hp_reg_pending)

<pre> reg [1:0] mclk_div; reg mclk; always @(posedge clk50 or negedge rst_n) begin if (!rst_n) begin mclk_div <= 2'd0; mclk <= 1'b0; end else begin mclk_div <= mclk_div + 1'b1; if (mclk_div == 2'd1) begin mclk_div <= 2'd0; mclk <= ~mclk; end end end end assign aud_xck = mclk; </pre>	<pre> reg [3:0] bclk_div; reg bclk; always @(posedge clk50 or negedge rst_n) begin if (!rst_n) begin bclk_div <= 4'd0; bclk <= 1'b0; end else begin bclk_div <= bclk_div + 1'b1; if (bclk_div == 4'd7) begin bclk_div <= 4'd0; bclk <= ~bclk; end end end end assign aud_bclk = bclk; </pre>
--	---

Đây là sơ đồ nối dây để có được xung 3.125Mhz (BCLK) bằng cách chạy đếm 8 xung của 50MHz mới thay đổi mức logic 1 lần, sau 16 lần là hoàn thành 1 chu kỳ của 'BLCK' để dịch âm thanh ra bằng wm8731, xung 12.5MHz (MCLK) tương tự mạch trên thiết kế cho đếm

đến 2 mới thay đổi mức logic 1 lần, sau 4 lần hoàn thành 1 chu kỳ của ‘MCLK’ là xung nguồn để hoạt động con chip âm thanh Woflson,

Sau khi có nguồn xung nuôi và dịch bit thì chúng em sử dụng giao thức I2C để giao tiếp cũng như ra lệnh cho con chip âm thanh hoạt động với cấu trúc dữ liệu truyền đi được cấu tạo như sau:

<pre>function [15:0] wm_pack; input [6:0] reg7; input [8:0] data9; begin wm_pack = {reg7, data9[8], data9[7:0]}; end endfunction function [15:0] wm_hp_both_word; input [6:0] vol; begin</pre>	<pre> wm_hp_both_word = wm_pack(7'h02, {1'b1, 1'b0, vol}); end endfunction function [15:0] wm_line_both_word; input [4:0] gain_code; begin wm_line_both_word = wm_pack(7'h00, {1'b1, 1'b0, 2'b00, gain_code}); end endfunction</pre>
--	--

Sau khi cài đặt xong thì bằng giao tiếp I2C chạy nhóm chuỗi 10 lệnh khởi động, reset chip(7'h0F), cấp nguồn(7'h06), chọn ngõ ra DAC(7'h04), báo cho chip biết là định dạng âm thanh là I2S_24bit (7'h07). Khi dây báo xong init_done được đẩy lên mức logic [1], thì con chip Woflson thực sự sẵn sàng. Cờ ‘finish_flag’ được bật lên [1] khi giao thức I2C truyền xong 1 lệnh, nhưng xung 50MHz quá nhanh để đọc cái cờ báo ở xung 2.5kHz nên dùng một thủ thuật bắt sườn bằng 1 nhịp qua D_FF với ngõ ra ‘finish_d’, bằng cách đọc giá trị (finish_d&&finish_flag) chỉ bắt đúng khoảnh khắc thay đổi nhưng chưa dịch bit qua D_FF.

Vì giao thức I2C chạy trên xung nhịp rất chậm nên nếu lỡ trường hợp chúng em chỉnh thông số quá nhanh, chưa kịp để I2C truyền xong thì sẽ gây ra hiện tượng hụt dữ liệu, để tránh tình trạng đó thì giải pháp nhóm em đưa ra là thêm vào một dây báo rảnh cho giao thức I2C bằng dây ‘hp_reg_pending’

```
// latch latest requests
if (hp_vol_update) begin
```

```

hp_shadow    <= hp_vol_code; // Cất vào bản nháp
hp_req_pending <= 1'b1;      // Giờ chờ "Có dữ liệu"
end

```

Register ‘hp_shadow’ sẽ đóng vai trò như một biến lưu trữ giá trị nhận từ uart mới khi giá trị I2C cũ chưa dịch xong, khi nào giá trị cũ được đẩy đi xong thì I2C sẽ lấy giá trị từ chỗ này để tiến hành dịch tiếp.

Đẩy lệnh đi vào con chip wm8731 bằng câu lệnh sau với cờ báo ‘init_done’ và biến đếm trạng thái ‘init_idx’:

<pre> if (!init_done) begin ctrl_busy <= 1'b1; // báo bận if (!ignition) begin mux_input <= init_word[init_idx]; ignition <= 1'b1; end // nạp lệnh để gửi đi if (!finish_d && finish_flag) begin ignition <= 1'b0; </pre>	<pre> if (init_idx == INIT_N-1) begin init_done <= 1'b1; init_idx <= 4'd0; current_op <= OP_NONE; end else begin init_idx <= init_idx + 1'b1; end end // kiểm tra nếu chưa đủ trạng thái thì nạp tiếp </pre>
---	--

Sau khi nạp xong 10 lệnh khởi động, cài đặt con chip âm thanh Woflson wm8731 thì cờ báo ‘init_done’ được bật lên mức logic [1] nhảy sang nhánh khi đã khởi động xong với cấu hình như sau:

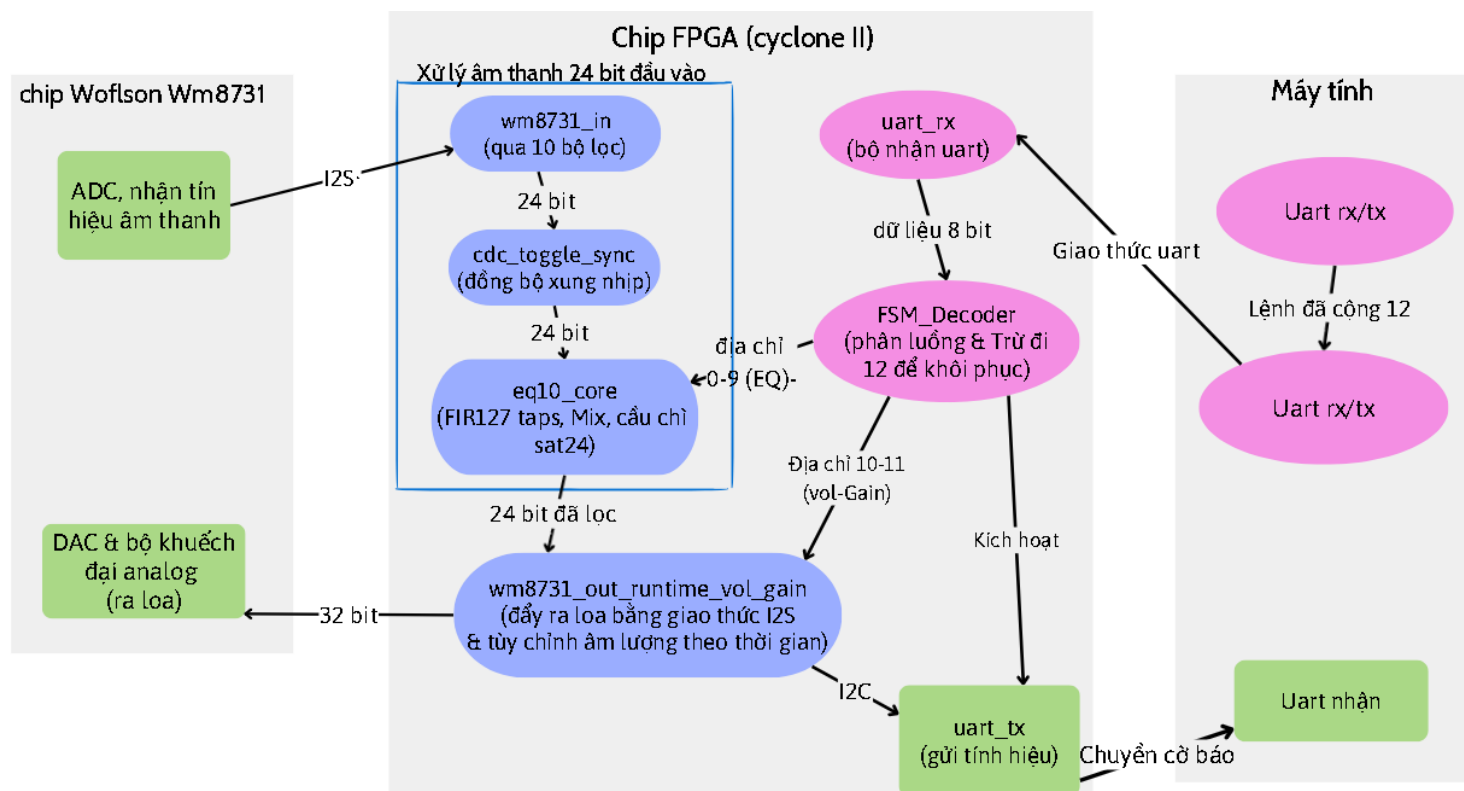
<pre> end else begin // priority: volume first, then line gain if (!ignition) begin if (hp_req_pending) begin mux_input <= wm_hp_both_word(hp_shadow); ignition <= 1'b1; hp_req_pending <= 1'b0; ctrl_busy <= 1'b1; </pre>	<pre> ignition <= 1'b1; line_req_pending <= 1'b0; ctrl_busy <= 1'b1; current_op <= OP_LINE; end else begin ctrl_busy <= 1'b0; current_op <= OP_NONE; end </pre>
--	---

<pre> current_op <= OP_HP; end else if (line_req_pending) begin mux_input <= wm_line_both_word(line_shadow); </pre>	<pre> end else if (!finish_d && finish_flag) begin ignition <= 1'b0; ctrl_busy <= 1'b0; current_op <= OP_NONE; end end end end </pre>
--	---

Vì ‘sample_in’ là cái biến ngõ vào âm thanh 24 bit đã được chỉnh, cấu hình tùy chỉnh ở trong module ‘eq10_core’ rồi nên ở module này sẽ thực hiện nhiệm vụ cài đặt cho chip âm thanh để tiến hành dịch 24 bit theo thời gian thực của âm thanh ra loa. Nhưng ở module này thì lại là 2 mức còn lại là Volume và Gain trong dải tùy chỉnh với mức ưu tiên trước hết là biên độ của âm thanh (volume), nếu nhìn tổng quan thì nó sẽ chạy kiểm tra vòng vô hạn những biến báo có dữ liệu ở các kênh (hp_reg_pending\volume hay line_reg_pending\Gain của mic) để thực thi lệnh.

Chương III. KIẾN TRÚC TỔNG THỂ VÀ TRIỂN KHAI TRÊN KIT DE2

3.1 Sơ đồ kết nối tổng quát toàn hệ thống (Top-level lab3_eq10_uart)



Kiến trúc tổng thể của hệ thống Equalized 10 dải được chia thành 3 khối vật lý chính: Khối mã hóa, giải mã âm thanh, khối giao tiếp máy tính và khối xử lý trung tâm. Hoạt động của hệ thống được chia thành 2 luồng dữ liệu độc lập, nhằm đảm bảo tính thời gian thực và tính đúng của âm thanh.

Âm thanh được truyền từ máy tính qua cổng Gain đi vào Kit DE2, đầu tiên nó đi vào con chip xử lý âm thanh Woflson wm8731 để số hóa thành dữ liệu 24 bit, thông qua giao thức dịch bit I2S đẩy vào chip xử lý FPGA. Đến đây nó đi qua module `wm8731_in` để cho qua 10 bộ lọc để phân thành 10 dải và chuẩn bị 24 bit dữ liệu mẫu để phục vụ cho những tác vụ sau. Vì cờ báo của bộ xử lý âm thanh ở con chip Woflson rất chậm nên cần đi qua một bộ đồng bộ xung nhịp (`cdc_toggle_sync`) để tránh đọc sai tín hiệu.

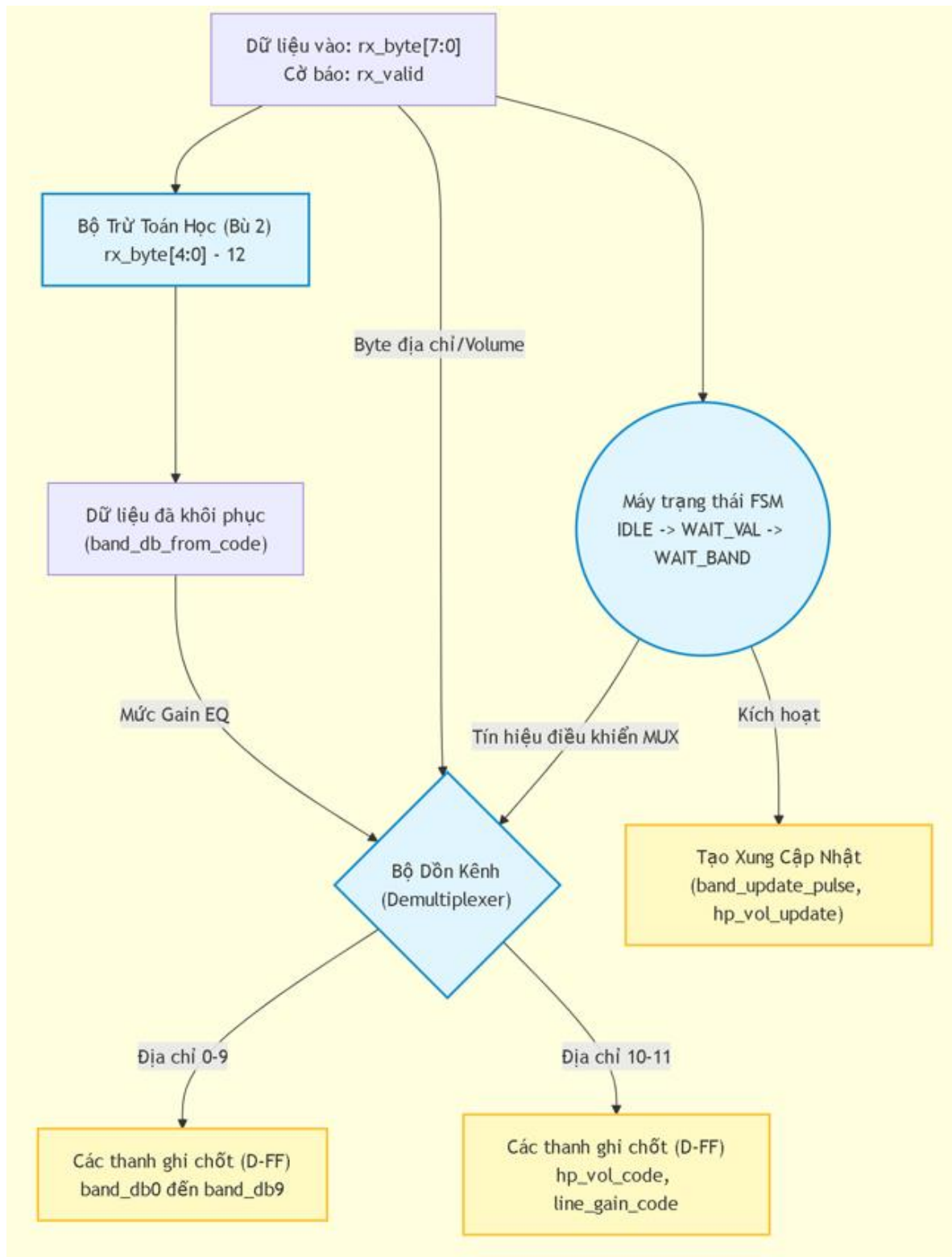
24 bit âm thanh sau đó đi đến module `eq10_core`, tại đây thực hiện bộ lọc FIR 127 taps với hệ số và địa chỉ nhận được từ Uart tiến hành nhân và cộng tuần tự 10 xung, kết quả cho qua một bộ giới hạn bão hòa bit với ngõ ra được định dạng 32 bit (8bit 0_LSB) tại module

wm8731_out_runtime_vol_gain và dịch bit qua bộ DAC của chip Wolfson thông qua giao thức I2S để tiến hành phát ra loa.

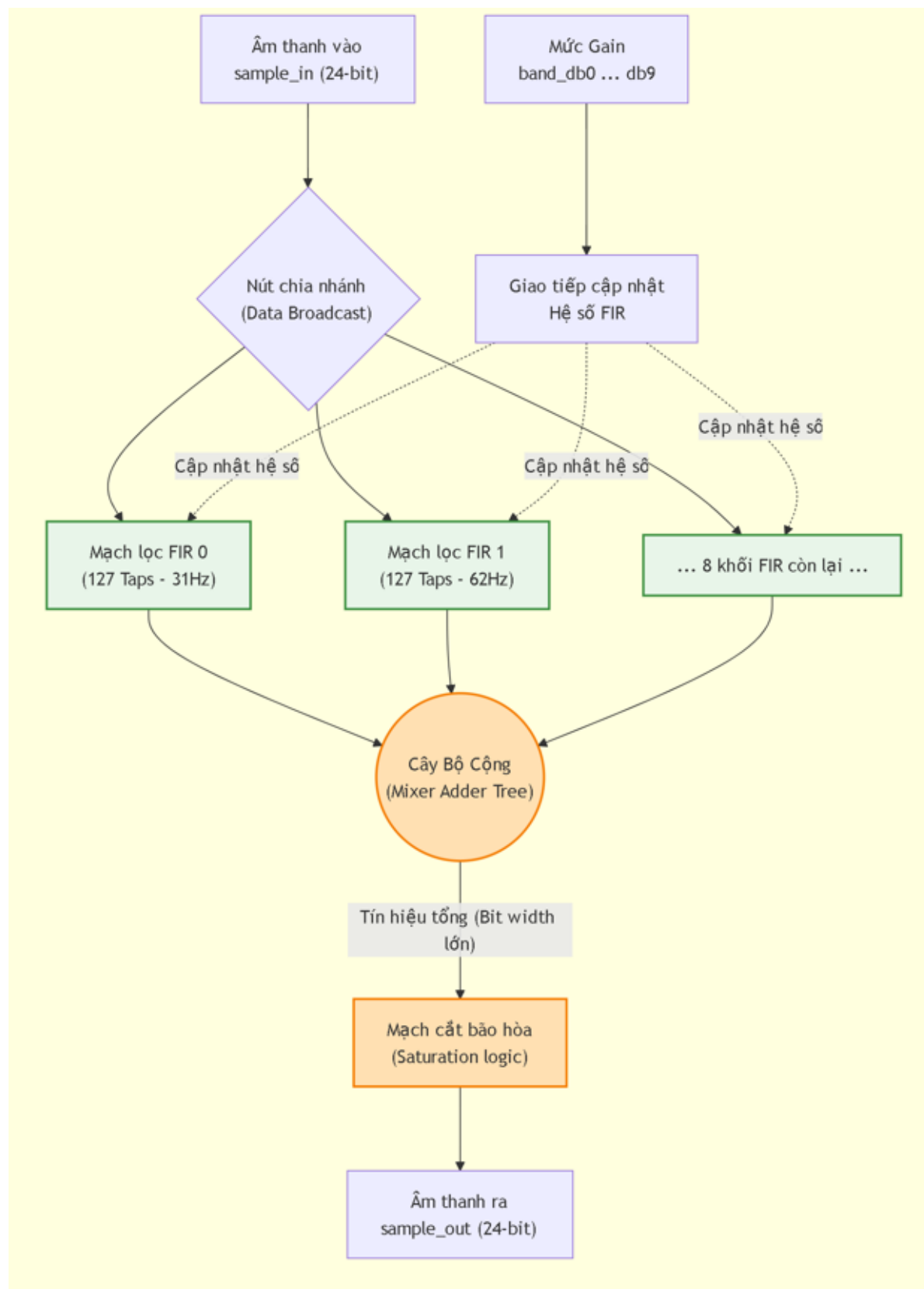
Luồng dữ liệu điều chỉnh âm thanh từ máy tính (EQ, Vol, Gain) đã được cộng thêm 12 để tiến hành gửi đến FPGA thông qua uart, Khi uart_rx nhận dữ liệu và chuyển đến máy trạng thái FSM_Decoder, ở khối trạng thái này nó sẽ trừ đi 12 để trả lại giá trị cũ, sau đó dựa vào bit địa chỉ ở trong dữ liệu nhận được chuyển đến module eq10_core (nếu địa chỉ từ 0-9) để cập nhật lại hệ số của bộ lọc FIR. Còn những trường hợp còn lại sẽ chuyển đến module wm8731_out_runtime_vol_gain để tiến hành tùy chỉnh giá trị biên độ thông qua giao thức I2C để thực thi lệnh âm thanh trên con chip âm thanh Wolfson.

3.2 Sơ đồ khối của các module thành phần

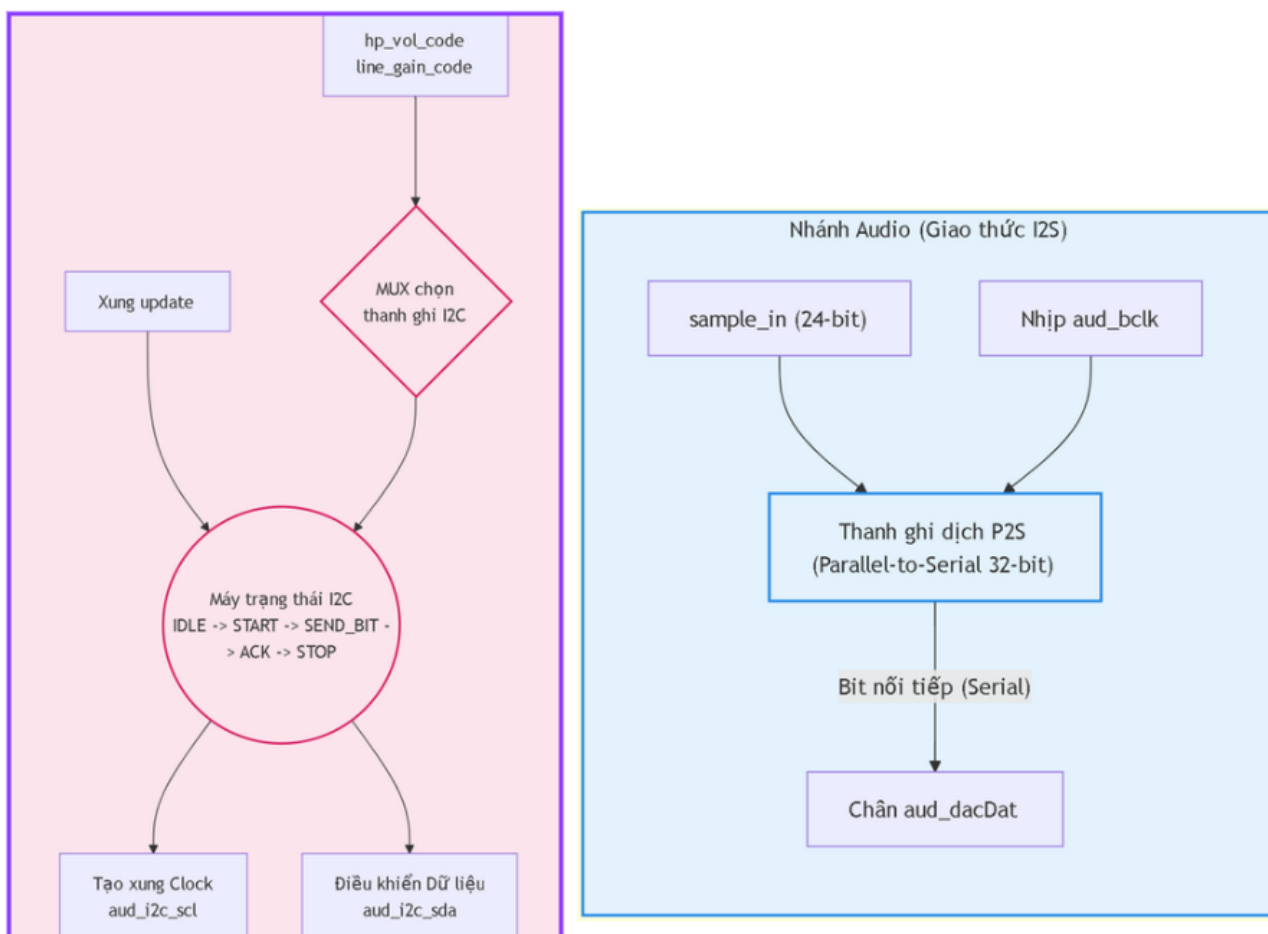
3.2.1 Khởi nhận và giải mã lệnh Uart (FSM_Decoder)



3.2.2 Khối xử lý dải âm (eq10_core)



3.2.3 Khối giao tiếp chip âm thanh (wm8731_out_runtime_vol_gain)



3.3 Cấu hình tài nguyên và gán chân tín hiệu trên kit Altera DE2

Hướng	Tên chân	Chân FPGA (PIN)	Chức năng chi tiết trong hệ thống
Input	clk50	PIN_N2	Cấp xung nhịp hệ thống (50MHz) cho toàn bộ board FPGA hoạt động.
Output	led_stt	PIN_AE22	Đèn LED báo hiệu trạng thái (nhấp nháy để xác nhận hệ thống đang chạy).
Input	rst_n	PIN_G26	Nút nhấn khởi động lại hệ thống (Reset phần cứng, tích cực mức thấp).
Output	aud_adc_lrck	PIN_C5	Xung phân kênh Trái/Phải (L/R Clock) báo nhịp cho khối thu âm (ADC) của WM8731.
Input	aud_adcDat	PIN_B5	Nhận luồng dữ liệu âm thanh số thô (chuẩn I2S) từ chip WM8731 đưa vào FPGA.
Output	aud_bclk	PIN_B4	Xung nhịp bit (Bit Clock) đồng bộ hóa việc truyền nhận từng bit âm thanh I2S.

Output	aud_dacDat	PIN_A4	Đẩy luồng dữ liệu âm thanh đã qua bộ lọc EQ (I2S) trở lại chip WM8731 để phát ra loa.
Output	aud_dacLrck	PIN_C6	Xung phân kênh Trái/Phải (L/R Clock) báo nhịp cho khối phát âm thanh (DAC) của WM8731.
Output	aud_i2c_scl	PIN_A6	Xung nhịp đồng bộ giao thức I2C dùng để cấu hình chip Wolfson.
Output	aud_i2c_sda	PIN_B6	Đường truyền dữ liệu I2C để gửi các lệnh cài đặt âm lượng (Master Volume, Line Gain).
Output	aud_xck	PIN_A5	Cấp xung nhịp chủ (Master Clock) từ FPGA sang để chip âm thanh WM8731 có thể hoạt động.
Input	uart_rx_pin	PIN_J22	Nhận các byte lệnh điều khiển (EQ, Volume) từ giao diện phần mềm máy tính (C#).
Output	uart_tx_pin	PIN_E25	Trả tín hiệu phản hồi (Ký tự 'K') về máy tính để xác nhận FPGA đã thi hành xong lệnh.

Chương IV. KẾT QUẢ THỰC NGHIỆM VÀ NHẬN XÉT

4.1 Kết quả biên dịch và phân tích tài nguyên phần cứng

Sau khi hoàn thiện mã nguồn mô tả phần cứng cho hệ thống xử lý âm thanh sử dụng bộ lọc FIR, quá trình tổng hợp và biên dịch đã được thực hiện thành công trên phần mềm Quartus II trên chip FPGA Altera Cyclone II như sau:

The screenshot displays the Quartus II IDE interface. The 'Flow Summary' window shows the compilation status as 'Successful' on April 24, 2026. It lists various metrics for the design 'lab3_eq10_uart' on a Cyclone II EP2C35F672C6 device. The 'Messages' window at the bottom shows several warnings and one success message.

Task	Time
Analysis & Synthesis	00:00:47
Fitter (Place & Route)	00:01:09
TimeQuest Timing Analysis	00:00:04
EDA Netlist Writer	
Gate Level Simulation	

Flow Summary	Flow Status
Flow Status	Successful - Fri Apr 24 09:15:56 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 S1 Web Edition
Revision Name	lab3
Top-level Entity Name	lab3_eq10_uart
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	9,687 / 33,216 (29 %)
Total combinational functions	7,990 / 33,216 (24 %)
Dedicated logic registers	3,977 / 33,216 (12 %)
Total registers	3977
Total pins	13 / 475 (3 %)
Total virtual pins	0
Total memory bits	0 / 483,840 (0 %)
Embedded Multiplier 9-bit elements	70 / 70 (100 %)
Total PLLs	0 / 4 (0 %)

Type	ID	Message
Warning	332102	Design is not fully constrained for setup requirements
Warning	332102	Design is not fully constrained for hold requirements
Success	293000	Quartus II 64-Bit TimeQuest Timing Analyzer was successful. 0 errors, 4 warnings
Success	293000	Quartus II Full Compilation was successful. 0 errors, 1356 warnings

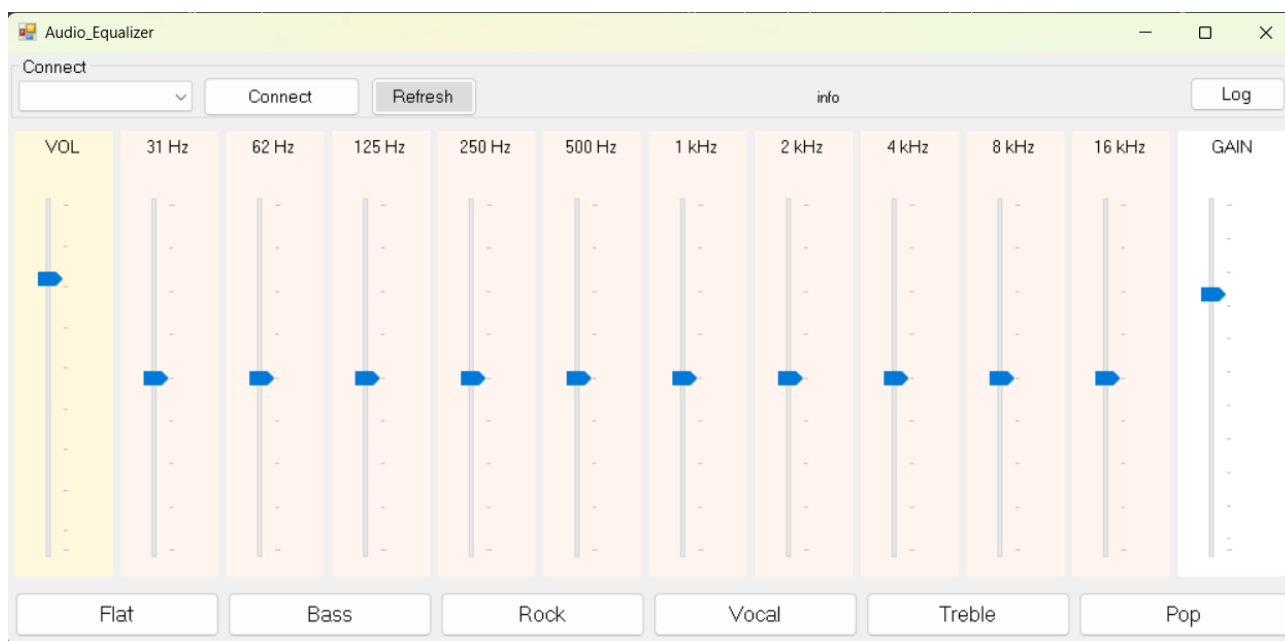
Hình 4.1: kết quả biên dịch chương trình

Dựa vào hình ảnh kết quả biên dịch thì hệ thống sử dụng 9,678 / 33,216 LEs, chiếm 29%. Trong đó số lượng các hàm tổ hợp là 7,990. Điều này cho thấy thuật toán điều khiển và lọc của hệ thống khá tối ưu, phần không gian logic còn dư nhiều để có thể nâng cấp thêm những tính năng khác trong những bài sau.

Tài nguyên thanh ghi tiêu tốn 12% của hệ thống (3,977 / 33,216 thanh). Số lượng thanh ghi này chủ yếu phục vụ cho việc tạo ra các đường trễ tín hiệu trong cấu trúc của bộ lọc FIR, hoặc phục vụ cho việc ép xung cho Pipeline để đảm bảo hệ thống đạt được xung nhịp tối ưu.

Hệ thống nhóm em thực hiện sử dụng 13 chân trên kit DE2. Số lượng chân này đáp ứng đủ các giao tiếp ngoại vi gồm bộ dao động thạch anh, nút reset, RX/TX cho giao thức Uart giao tiếp với máy tính.

Hệ thống sử dụng 100% tài nguyên bộ nhân 9bit (70/70 bộ). phản ánh tính chất của bộ lọc FIR 127 taps để đạt được độ dốc cắt tần số tốt và đáp ứng pha tuyến tính đảm bảo chất lượng âm thanh tốt nhất có thể. Đòi hỏi nhiều phép nhân tích chập đồng thời giữa tín hiệu đầu vào và hệ số lọc.



Hình 4.2: giao diện người dùng bằng C#

4.4 Nhận xét hạn chế và hướng phát triển hệ thống

Mặc dù nhóm thực hiện đề tài xử lý tín hiệu âm thanh trên kit thí nghiệm DE2 điều chỉnh biên độ của từng dải tần số cụ thể nhưng xử lý với tần số clock chưa được tốt, hạn chế của nhóm là không sử dụng tốt kỹ thuật Block to block để nâng xung nhịp xử lý của hệ thống lên (>50Mhz).

Giao diện người dùng bằng ngôn ngữ C# chưa được tốt vì thanh trượt điều chỉnh vó hệ số chưa được lớp lý dẫn đến hiện tượng trượt chưa hết thanh mà đã đạt giá trị Min hoặc Max trước đó.

Do sử dụng nhiều resistor và nhiều khối tính toán bằng Pipelining nên tín hiệu từ ngõ vào đến ngõ ra loa nghe được sẽ bị trễ một khoảng thời gian tương đối, nhưng vẫn có thể chấp nhận được. Tín hiệu âm thanh được lượng tử hóa bằng dấu chấm tĩnh nên có độ sai số rõ rệt hơn so với bộ lượng tử bằng dấu chấm động nên âm thanh chưa đạt được chất lượng tối đa.

Hướng phát triển

Thay vì dùng N bộ nhân cho N-taps, hệ thống tương lai sẽ tận dụng tính đối xứng của các hệ số bộ lọc FIR. Bằng cách cộng các mẫu dữ liệu lại trước khi nhân, ta có thể giảm tương đối số lượng bộ nhân cần thiết, tiết kiệm triệt để tài nguyên.

Thay vì xử lý toàn bộ âm thanh ở tần số thấp, có thể áp dụng kỹ thuật Decimation (giảm tốc) để xử lý các dải tần tần số thấp, sau đó Interpolation (tăng tốc) trở lại. Cách này giúp giảm khối lượng tính toán.

